Google Summer of Code 2026

Online Store Extension

Real-Time Overlay for E-Commerce

Large (350 hours)

Mentors

Eloïse Prévôt, Sandrine Comeau, Louis Bastarache

Contributor

Name	Sam K Thampan
Email	kthampansam18@gmail.com
OpenFoodFacts ID	samkthampan
Github Username	devsk18
Slack ID	U0AF4LYBKD5
Timezone	IST (UTC+5:30)

Table of Contents

1. Project Overview **3**
Project Benefits to Open Food Facts and its Community 3
2. Proposal Overview **3**
Proposed Solutions 3
3. Technologies **5**
4. Extension Workflow **6**
5. Implementation Details **9**
Core Architecture 9
1. Orchestrator 9
2. Site Adapter 9
3. Adapter Registry 9
Product Matching 9
1. Barcode 9
2. Store Product ID 9
3. Search API 12
Core Services 13
1. Storage & Caching 13
2. Messaging 13
3. Internationalization (i18n) 13
4. Settings Service 13
UI & Data 14
1. Popup UI 14
2. Banner UI 14
Preference Scoring 15
6. Timelines & Milestones **16**
April (Pre-GSoC) 16
Community Bonding Period (May 1 – May 24) 16
Phase 1 - Foundation & Core Services (week 1 - 2) 17
Phase 2 - Core Architecture & Walmart Adapter (week 3 - 5) 17
Phase 3 - Popup UI & Banner Components (week 6 - 8) 17
Midterm Evaluation (week 7) 18
Early Alpha Testing & Feedback (week 8) 18
Phase 4 - Additional Adapters & Testing (week 9 - 12) 18
Phase 5 - Canadian DuckDB API (week 13 - 15) - (If available) 18
Beta Testing & Feedback (week 16) 19
Final Buffer 19
Final Evaluation 19
7. Background & Commitment **19**
Why I am the Right Person 19
How I Plan to Achieve Completion 20
Time Commitments 20
Open Source Contributions 20
Prototyping Experience 21
Relevant Past Projects 21

1. Project Overview

Grocery shoppers rarely have the time or tools to evaluate the nutritional and environmental impact of products while shopping online. This project addresses that gap by building a cross-browser extension that surfaces Open Food Facts data directly on Canadian grocery stores' webpages, helping users make more informed and healthy choices without leaving their shopping experience.

Project Benefits to Open Food Facts and its Community

This project directly extends the value of the OFF dataset by bringing it into the grocery shopping experience of Canadian users. Shoppers get access to nutritional and environmental data at the point of purchase, without leaving their browser. In return, every product lookup and contribution drives traffic back to the OFF platform, growing its user base and community.

2. Proposal Overview

DriveOFF and **Nutribanner** are previous versions of this idea, which failed due to the following limitations.

1. **Product Matching Strategy**: Both extensions relied only on barcodes embedded in the DOM to identify products. No fallback existed when a barcode was absent.
2. **API Efficiency**: They used the Product API without field filtering, pulling far more data than needed, resulting in unnecessary data transfer and higher memory usage.
3. **Caching**: Browser HTTP cache was used to store API responses. This led to stale data persisting indefinitely with no expiry or invalidation strategy.
4. **Performance**: API calls were fired in a fan-out pattern. On a product listing page with 50 items, 50 separate calls were made, regardless of whether the user ever looked at those products.
5. **UI & Insights**: The interfaces were outdated and not responsive. Lack of descriptions made it harder for shoppers to understand what the scores meant.
6. **Features**: Neither extension offered functionality that meaningfully improved the shopping experience beyond displaying product data.

Proposed Solutions

The following solutions are designed to achieve the key objectives of the project, with a heavy focus on performance and scalability to support

Product Matching Strategy

Most Canadian grocery e-stores expose a barcode and a store-specific product ID on product pages, either in the DOM or through network calls. As users visit these pages, the extension captures both identifiers and uses them to build a product mapping in our database and the user's browser (so on listing pages, products with an existing mapping can

be resolved instantly without an API call). Until a mapping exists, the Search API serves as the fallback.

This avoids bot-based crawling, which most retailers actively block. Extracting data passively through real user visits makes the approach far more reliable. The mapping database and its supporting APIs will be done as part of the Canadian DB.

API Efficiency

The Product API's fields parameter limits responses to only what the request needs, cutting unnecessary data transfer and reducing browser memory usage.

Caching

A lightweight cache service is built on top of WXT's browser storage API. Each cached entry carries an expiry timestamp, checked on every read to either return or invalidate the data. A background script runs on extension load to clear expired entries proactively. This reduces API calls significantly and keeps the data reasonably fresh (for incomplete products).

Performance

API calls are made only for products visible in the viewport. Products outside it are observed and fetched only when they scroll into view. This reduces API calls, avoids hitting api rate limits and server overload.

UI & Insights

Standard OFF computed symbols (Nutri-Score, NOVA, Eco-Score) will be retained, and nutrient level indicators for fat, saturated fat, sugar, and salt will be added. Each icon is accompanied by a short description, so shoppers know exactly what they are looking at.

Suggested Features

- **Preference Scoring**: The extension can include a user preference-based scoring system, similar to the OFF’s Personal Search Feature, which helps shoppers evaluate products faster. User preferences will be stored locally for privacy.
- **Customizable UI**: Users can toggle UI settings to control banner appearance and switch between languages directly from the extension.

3. Technologies

Choices

Language	TypeScript
Framework	WXT (Web Extension Toolkit)
UI	Preact
Styling	Tailwind CSS
APIs	OFF Product API, Search-a-licious, Canadian DB API
Testing	Vitest, playwright
CI/CD	GitHub Actions + WXT Publishing
Docs & Tracking	GitHub Pages + GitHub Projects

Reasons

1. TypeScript
    - Catches bugs at development, not runtime, Safer refactoring, IDE support
    - Easier for new contributors to understand data flow
    - Why not JavaScript - poor at catching bugs until runtime
2. WXT
    - Supports any frontend framework, has good built-in plugin support, and good documentation
3. Preact
    - Lightweight, hooks support, functional component reusability
    - Why not React/Vue/Svelte - heavy bundle size
    - Why not JS string-based templating: poor maintainability & readability
4. Tailwind CSS
    - Less boilerplate, utility-first, easier to keep UI consistent
    - Why not plain CSS - harder to manage and scale as styles grow
5. OFF Product API + Search-a-licious or Canadian DB API
    - Fallback for the Canadian DB API
    - If available, it helps in a custom product matching strategy
6. Vitest + playwright
    - Unit & integration testing, built-in WXT plugin support
7. GitHub Actions + WXT Publishing + GitHub Pages + GitHub Projects
    - Automated tests and publishing
    - Documentation and project tracking

Integrating analytics and error-tracking tools will help evaluate extension performance and runtime issues. (OFF already uses Matomo & Sentry)

4. Extension Workflow

Initialisation

When a user visits a supported grocery site, the content script loads and resolves a matching site adapter from the adapter registry. The adapter is passed to an orchestrator instance, which manages the entire flow from detection to rendering.

Product Detection

The orchestrator initializes a MutationObserver to watch for new nodes added to the page. It looks for two types of product templates using the structure and methods defined in the site adapter.

Product View Page

When a product page is detected, the adapter extracts the product data from the DOM. A Shadow Root is attached to the injection element at the position defined in the adapter, and the appropriate banner component is rendered.

Product List Page

When a product listing is detected, items visible in the viewport immediately follow the same rendering flow. Items outside the viewport are attached to an IntersectionObserver and follow the same flow once they enter the viewport.

Data Fetching

The banner component sends a message to the background service worker to fetch product data. The service worker asks the data layer to serve from cache if available, else falls back to the API. Product matching follows this priority order:

1. Barcode-based matching
2. Store-specific product ID (when Canadian DB is active)
3. Search query built from product name, brand, and package size

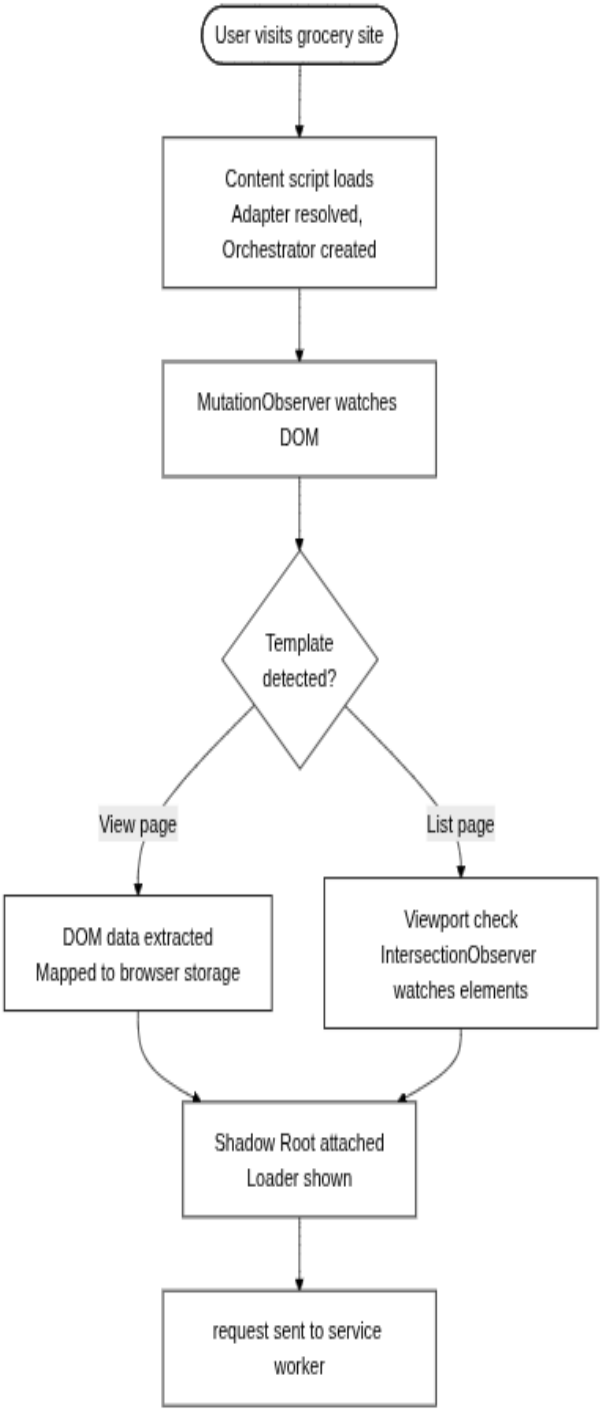
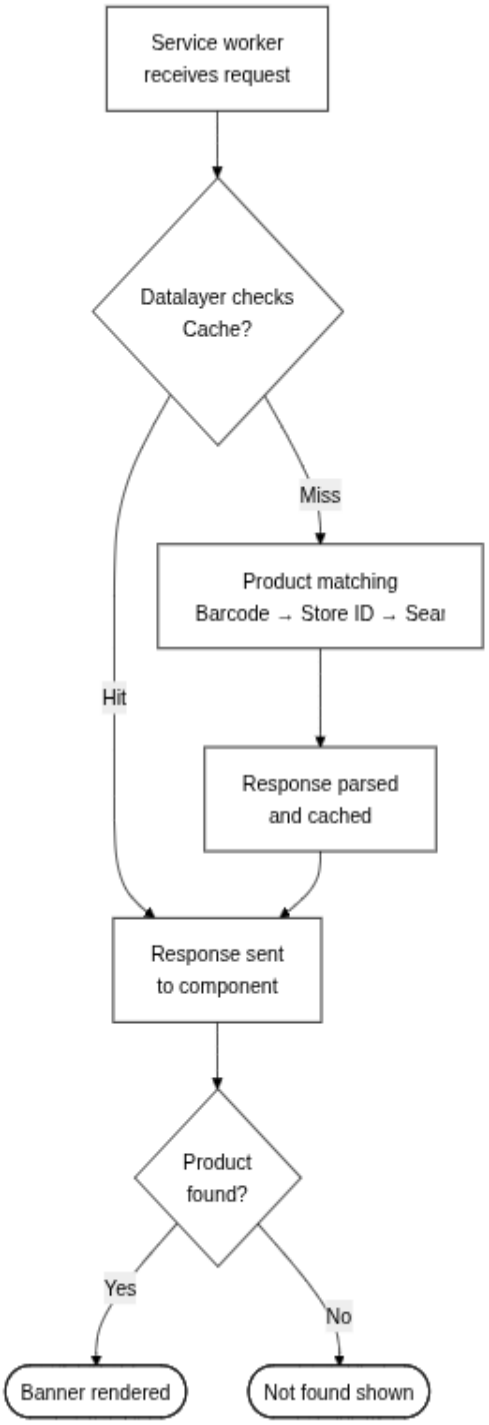
The response is parsed, cached, and sent back to the component. A loader is shown until data arrives. If data is found, the product banner is rendered. If not, a product not found state is shown.

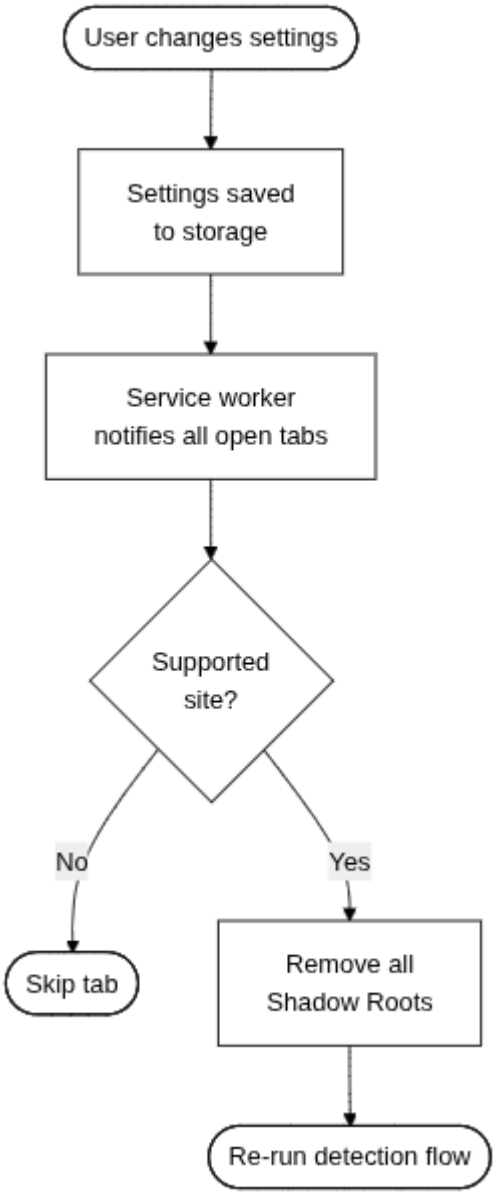
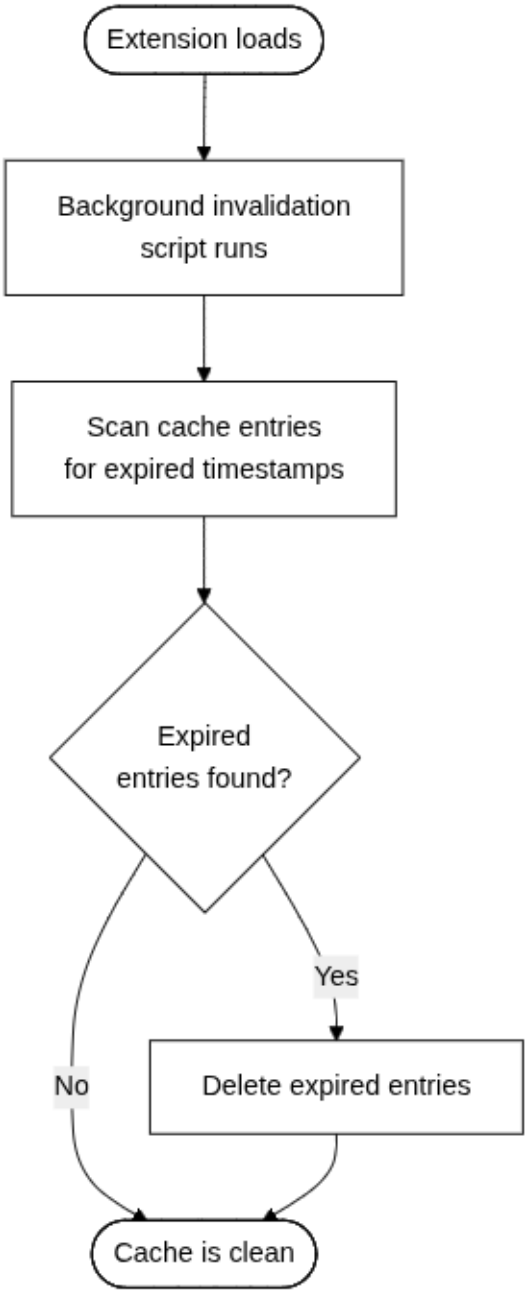
Settings & Re-render

When the user changes any settings, the service worker notifies all open tabs to re-render. All existing Shadow Roots are removed, and the full flow runs again with the updated settings without reloading the tab.

Preference Scoring

The preference score index is computed on extension install and on every change to preference settings. Each product's score is evaluated at render time.

Section 1: Detection & Setup	Section 2: Data Fetching & Rendering
	

Section 3: Settings & Re-render	Section 4: Cache Invalidation
	

5. Implementation Details

Core Architecture

The extension needs to support new store integrations without touching the existing codebase. This is achieved through three collaborating components.

1. Orchestrator

From a top-level view, the flow is the same for every site, which is DOM observation, product detection, and rendering coordination. The orchestrator handles this common flow and has no store-specific knowledge, which helps us to separate the flow from site adapters. Also helps us avoid bloating the content script.

2. Site Adapter

Isolates store-specific logic. A base abstract class defines the minimum contract every adapter must implement. Adding a new store means adding a new adapter file. Nothing else changes.

3. Adapter Registry

Maps hostnames to adapter classes. The content script resolves the correct adapter at runtime.

Product Matching

Identifying a product on a grocery site and mapping it to an OFF product is the most critical part of the extension. The matching follows a 3-tier waterfall; each tier is attempted in order, falling back to the next if unsuccessful.

1. Barcode

The most reliable match. When a user visits a product page, the adapter extracts the barcode from the DOM or network calls. The barcode is used to query the OFF Product API or Canadian DB API directly.

OFF Product Opener - Product API

```
GET /api/{version}/product/{barcode}
  ?cc={country}
  &lc={locale}
  &fields={field_names}
```

2. Store Product ID

A. Local Mapping
When a barcode is unavailable, the extension checks the local mapping built from previous visits to the product page. The store product ID is used to look up a previously resolved barcode. Mainly useful for product listing pages.

B. Server Mapping
Local mapping works well for returning users, but new users have no mapping data yet, so they fall back to search immediately. A server-level mapping solves the cold-start problem by sharing verified mappings across all users.

How it works
When a user visits a product page, the extension captures the store product ID and barcode and submits them to the server via the Mapping API. Each submission is tagged with an extension_id to track anonymous contributions. As more users visit the same product page, the submission count grows. Once it crosses a defined threshold, the mapping is marked as verified.

When any user needs to resolve a product, the extension first checks browser storage, then queries the server via the Lookup API. If a legitimate mapping is found, it is returned and cached locally for future use. If no mapping exists, the extension falls back to the search API.

Concerns
- **Data quality**: Incorrect mapping submissions from abusers can affect real users
- **Security & Abuse**: Anonymous submissions open the door to intentional mapping pollution
- **Anonymous trust**: Without authentication, submission quality depends entirely on rate limiting and majority voting

Solutions
- **Majority voting**: A mapping is marked verified only after a defined number of independent submissions agree on the same barcode and store product ID combination.
- **Rate limiting**: To prevent abuse, two rate-limiting rules can be enforced.
    1. Submissions from the same IP are capped at a defined limit per minute.
    2. The same barcode cannot be submitted with a conflicting store product ID within a short cooldown period by the same IP or Extension ID, preventing intentional mapping pollution.
    3. Redis is the recommended approach; it introduces an additional infrastructure dependency.

Post-GSoC Scope
The following are additional security layers that can be built on top of the initial implementation after the GSoC period:
- **Community flagging**: Incorrect mappings can be flagged and reviewed through a manual or automated verification layer. If Nutripatrol allows the integration, it can be utilized for the flagging system.
- **OFF authentication**: Submissions tied to OFF accounts carry a higher trust weight. Also drives new users to the OFF platform, growing the contributor base.

Product API

```
GET /api/{version}/product/{barcode}
  ?lc={locale}
  &fields={field_names}
```

Lookup API

```
GET /api/{version}/stores/mappings
  ?store_uuid={store_uuid}
  &store_product_id={value}
```

Combined Approach

It can reduce round-trip latency by resolving both the mapping and product data in one call.

```
GET /api/{version}/product
  ?lc={locale}
  &type={barcode|store_product_id}
  &value={type_value}
  &store_uuid={store_uuid}
  &fields={field_names}
```

Mapping API

```
POST /api/{version}/stores/{store_uuid}/mappings
{
  "store_product_id": "123456",
  "barcode": "0063100888053",
  "extension_id": "id-generated-on-install"
}
```

Database Design

The following table structure is suggested in SQL, considering the Canadian DB will be built on DuckDB.

stores

Column	Type	Notes
store_uuid	VARCHAR	UUID - for unique identification
store_name	VARCHAR	e.g. Walmart Canada

product_store_mappings

Column	Type	Notes
id	INTEGER	Primary key
store_uuid	VARCHAR	References stores(store_uuid)
store_product_id	VARCHAR	Store-specific product ID
barcode	VARCHAR	OFF barcode
submission_count	INTEGER	Trust score, default 1
is_verified	BOOLEAN	Default false

created_at	TIMESTAMP	
updated_at	TIMESTAMP	

mapping_submissions

Column	Type	Notes
id	INTEGER	Primary key
mapping_id	INTEGER	References product_mappings(id)
extension_id	VARCHAR	Unique ID generated on install
submitted_at	TIMESTAMP	

DuckDB's indexing techniques can be utilized for additional performance optimization on frequently queried columns, such as store_uuid and store_product_id.

3. Search API

If no mapping exists, a search query is built from the product name, brand, and package size and sent to the Search-a-licious API or the Canadian DB API. The result will be shown with a warning indicating it may not be an exact match.

Search-a-licious - Search API

```
GET /search
  ?q={query}
  &langs={locale}
  &page_size={n}
  &fields={field_names}
```

DuckDB Search API

Based on prior experiments with DuckDB's full-text search and vector search on the OFF Canada dataset, the following approach is suggested as a Canada-specific alternative to Search-a-licious for a better search fallback tier.

```
GET api/{version}/search
  ?q={query}
  &page_size={n}
  &match_score={n}
  &fields={field_names}
```

Vector Search - Text

Full-text search has limited multilingual support, making it a weaker fit for our search. A multilingual sentence transformer model for vector search handles this naturally.
The approach:

- Generate embeddings using a sentence transformer model

    - paraphrase-multilingual-MiniLM-L12-v2 - 384 dim - lighter, faster, small storage footprint compared to 768 dim models
- Store them as a FLOAT[ ] array column in DuckDB
- Query using array_cosine_similarity for nearest neighbour search

Total products: 116,971 (may differ now)
Missing product_names: 2690 (2.3%)

Vector Search - Image
Image-based product matching was explored using OpenCLIP embeddings and vector similarity search. While it showed promising results, it was ruled out for the following reasons:

- Front images are not consistently available across all products in the OFF dataset, making a reliable image-to-image comparison difficult.
    - Total products: 116,971
    - Missing front image: 22,771 (19.47%)
- Transferring product images purely for matching adds network overhead.

Core Services

1. Storage & Caching

WXT's browser storage API is chosen over IndexedDB and other alternatives for its simplicity and built-in WXT integration. The browser storage provides up to 10MB by default, extendable with the unlimitedStorage permission, which is sufficient for the current scope due to regular cache invalidations, keeping the storage lean.

2. Messaging

The content script and background service worker communicate via WXT's browser.runtime messaging polyfill. This ensures cross-browser compatibility without writing separate implementations for different browsers.

3. Internationalization (i18n)

EN/FR support is a core requirement for Canadian Projects. A custom lightweight JSON translation manager will be used over the browser's native i18n API and third-party i18n libraries. Because the browser's native i18n API ties language selection to the browser's own language setting, with no support for live switching.

4. Settings Service

The settings service manages all user preferences, like score icon visibility, language preference, and preference scoring weights. Settings are persisted using the storage service and propagated to all open tabs via the service worker when changed.

UI & Data

1. Popup UI

The pop-up serves as the main settings interface for the extension. It allows users to:

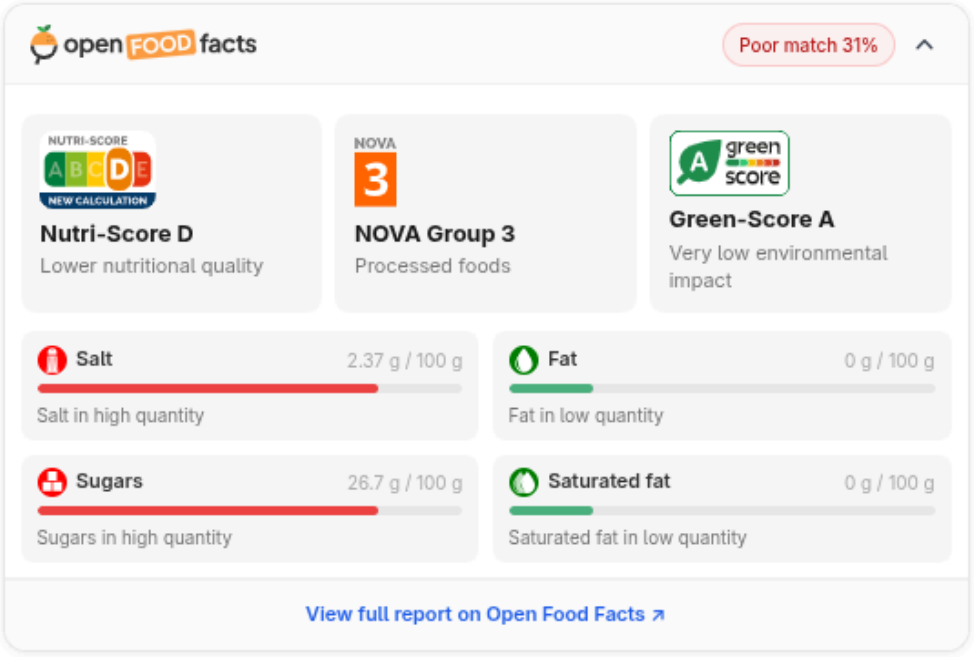
- Toggle score icon visibility
- Set preference scoring weights per attribute
- Switch between different languages
- View a quick summary of the current preference score configuration

The pop-up communicates with the service worker via the messaging layer, so any settings change triggers a re-render across all open tabs with the updated configuration.

2. Banner UI

The extension renders UI components inside a Shadow Root attached to an injection element defined in each site adapter. Shadow DOM ensures complete style isolation, restricting the extension's styles from conflicting with the grocery site's stylesheet, and vice versa.

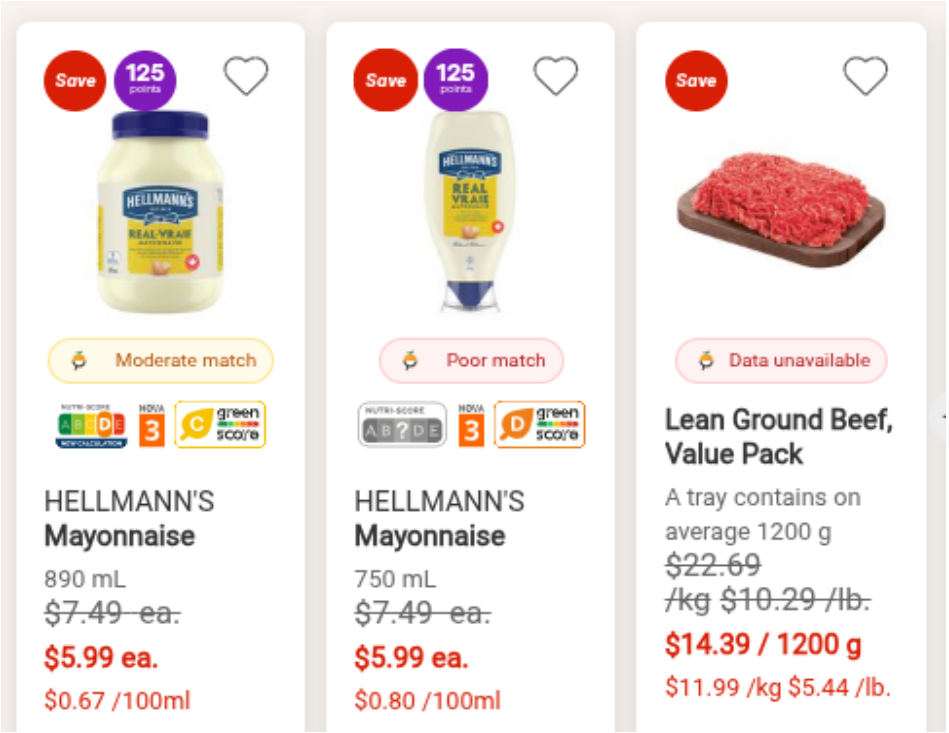
(Prototype)



Preact is used to render components inside the Shadow Root, keeping the bundle lightweight. Three core component states are handled:

- **Loading**: a loading animation is shown while data is being fetched.
- **Found**: Nutri-Score, NOVA, Eco-Score icons with short descriptions, nutrient level indicators for fat, saturated fat, sugar, and salt, and the preference score badge are shown, along with a link to the full product page on the OFF website.
- **Not found**: a warning is shown with a link to contribute the missing product to OFF.

On listing pages, a compact banner version is rendered with a preference score, and a warning is shown when the result comes from the search fallback tier.



Preference Scoring

The preference scoring system helps shoppers evaluate products against their personal dietary preferences at a glance. It is inspired by the OFF personal search system. For the Extension Simplicity, preference scoring will be limited to the following attributes:

1. **Score**: Nutri-Score, NOVA Group, Eco-Score
2. **Nutrition**: Fat, Saturated Fat, Sugar, Salt

Importance Level - Factor mapping

Importance	Factor
not_important	0
important	1
very_important	2
mandatory	4

Grade - Score mapping

Grade	Score
a	+2
b	+1
c	0
d	-1
e	-2

How it works

Each attribute is assigned an importance level by the user - not_important, important, very_important, or mandatory, which maps to a factor of 0, 1, 2, or 4, respectively. At the same time, each attribute grade maps to a base score ranging from +2 (grade a) to -2 (grade e).

score = sum(grade score × importance factor)
maxScore = sum of (2 × factor) for all active attributes
percentage = ((score + maxScore) / (2 × maxScore)) × 100

Precomputed Index Table
Rather than recalculating weights on every product render, a scoring table and maxScore are precomputed from user settings and stored in the browser. It is built on install and rebuilt on every change to preference settings.

Real-Time Score Evaluation
At render time, the product's data is evaluated against the precomputed table. The result is a percentage score and a compatibility flag.

A product is marked:

- Not compatible: If any mandatory attribute scores negatively
- Unknown match: If any mandatory attribute has no data
- Compatible: If all mandatory attributes pass
    - < 50% - Poor Match
    - 50 to 69% - Moderate Match
    - 70 to 89% - Good Match
    - ≥ 90% - Very Good Match

6. Timelines & Milestones

April (Pre-GSoC)

- Analyze & Prototype additional site adapters
- Experiment with different UI layouts and banner designs
- Contribute to the OFF community repositories
- Experiment with a local DuckDB server
- Experiment with data extraction and matching strategies

Community Bonding Period (May 1 – May 24)

- Clarify and document all architecture decisions with the mentor
- Finalize UI/UX designs and component structure
- Communicate with the Canadian DB team to align on mapping API integration
- Set up development environment, CI/CD pipeline, and GitHub Projects board
- Agree on coding conventions, PR process, and milestone targets with mentor

Note: Each phase includes a buffer period to handle unexpected issues or delays. Any unused buffer time will roll over as a head start on the next phase, meaning smooth progress may result in an early completion than expected.

Phase 1 - Foundation & Core Services (week 1 - 2)

Goal: Build all reusable services upfront since they are discussed & known before implementation begins.

Deliverables:

1. WXT project setup with TypeScript, Preact, Tailwind CSS
2. Storage & Cache service with TTL and background invalidation script
3. Settings service with default settings
4. Messaging layer between the content script and the service worker
5. OFF Product API integration
6. Search-a-licious API integration
7. i18n service with EN/FR support
8. Preference scoring service
9. Unit tests for all services
10. Documentation on project setup and folder structure for future contributors

Milestone: All core services are functional and tested in isolation.
Buffer: (week 2) - code reviews, feedback iterations

Phase 2 - Core Architecture & Walmart Adapter (week 3 - 5)

Goal: Build the adapter system and validate the full flow end-to-end on one site.

Deliverables:

1. Base Adapter abstract class and data extraction strategies
2. Adapter registry with runtime resolution
3. Orchestrator with MutationObserver and IntersectionObserver
4. walmart.ca adapter - data extraction and injection flow
5. 3-tier product matching - barcode, local mapping, search fallback
6. Local mapping and server mapping submission API (using local server)
7. Unit tests for orchestrator, registry, and matching logic
8. Documentation on how to add a new grocery site

Milestone: Extension works on walmart.ca, detects products, matches them, serves and caches data.
Buffer: (week 5) - adapter or extraction issues, code reviews, feedback iterations

Phase 3 - Popup UI & Banner Components (week 6 - 8)

Goal: Deliver meaningful UI for product and pop-up pages.

Deliverables:

1. Shadow DOM setup with Tailwind CSS scoped injection
2. Product View Banner UI - Nutri-Score, NOVA, Eco-Score with descriptions and preference score
3. Nutrient level indicators UI - fat, saturated fat, sugar, salt
4. Product list view compact banner UI with preference score
5. Loader and not found states with the contribute link
6. Similar match warning for search fallback results

7. Popup UI and settings integration
8. UI customization - show/hide score icons, language switching
9. Documentation on UI and Preference Scoring

Milestone: Full end-to-end flow working on walmart.ca with all banners and pop-up UI
Buffer: (week 8) - Iterations based on mentors and community feedback

Midterm Evaluation (week 7)

Expected state: Core architecture, all services, Walmart adapter, and partial or full UI working end-to-end.

Early Alpha Testing & Feedback (week 8)

Following the midterm evaluation, the extension will be shared with a group of community members for early feedback. Runs in Parallel to the development phase.

Focus areas:

- Product matching accuracy on walmart.ca without Canada DB API
- Banner UI clarity and usefulness
- Preference scoring relevance
- General usability and performance

Feedback collected during this period will be used to prioritize fixes and improvements in the next phase.

Phase 4 - Additional Adapters & Testing (week 9 - 12)

Goal: Expand to more Canadian grocery sites and ensure reliability.

Deliverables:

1. Remaining grocery site adapters
2. Playwright integration tests for product matching and banner rendering
3. Full unit test coverage review
4. Performance profiling and optimization
5. Analytics & error tracking service integration (if any)
6. Documentation on how to download and set up in a user's browser.
7. GitHub Pages documentation

Milestone: Extension that works across the top Canadian grocery sites with full test coverage without DuckDB API
Buffer: (week 12) adapter issues, cross-browser testing, mentor review

Phase 5 - Canadian DuckDB API (week 13 - 15) - (If available)

Goal: Build and integrate the server-level mapping API into the Canadian DuckDB project.

Deliverables:

1. DuckDB table setup - stores, product_mappings, mapping_submissions
2. Mapping API - mapping submission endpoint
3. Lookup API - mapping lookup endpoint

4. Rate limiting layer - IP-based and barcode cooldown window
5. Majority voting logic for mapping verification
6. Integrate the Canadian DuckDB API into the extension data layer
7. Switch product matching to use Canadian DuckDB when available
8. Unit tests for all API endpoints

Milestone: Server-level mapping is live, submissions collected, and the extension resolves products through the Canadian DuckDB API
Buffer: (week 15) - API design iterations with the Canadian DuckDB team, initial setup and integration issues

Beta Testing & Feedback (week 16)

Goal: Validate the fully featured extension across multiple sites with real users before final submission.

Activities:

- Publish extension builds to stores.
- Share with OFF community members, volunteers, and personal networks

Collect structured feedback on:

- Product matching accuracy across different sites
- Banner UI clarity and usefulness
- Preference scoring relevance
- EN/FR language switching
- General performance and stability

Log all reported issues on GitHub and triage by priority
Address critical bugs before final submission

Milestone: Feedback collected, issues resolved, extension stable across all supported sites
Buffer: (Week 16) - issue triage, bug fixes, final polish

Final Buffer

Bug fixes, Code reviews, Documentations

Final Evaluation

Expected state: A fully functional cross-browser extension working across the top Canadian grocery sites, with complete test coverage, GitHub Pages documentation, and the Canadian DuckDB mapping API integrated and live.

7. Background & Commitment

Why I am the Right Person

I am a Software Engineer and MCA postgraduate from APJ Abdul Kalam Technological University with two years of industry experience. I recently stepped away from my full-time role to focus on learning, collaborating, and building meaningful software.

What I really bring to OFF:

- Proficiency in building scalable, secure and performance-oriented systems - reflected in decisions like viewport-based lazy loading, caching, and rate limiting
- Practical knowledge of REST API design conventions and best practices
- Expertise in collaborative development workflows - commit and PR conventions
- Problem-solving ability - product mapping and majority voting are key examples of my thought process.
- Understanding of the OFF ecosystem, features, and data through contribution and research.

How I Plan to Achieve Completion

The proposal is structured to de-risk delivery at every stage.

1. Core services are built first, so every subsequent phase has a stable foundation.
2. The first full site adapter is complete before the midterm, giving a working deliverable early.
3. The Canadian DuckDB integration is placed last, so the extension remains fully functional using OFF APIs even if DuckDB is delayed.

Daily development logs, weekly meetings, and small focused PRs keep progress visible and feedback loops short. Buffer periods are built into every phase to absorb unexpected issues without affecting the overall timeline.

Time Commitments

I am not currently committed to any organization or educational institution, making me fully available to dedicate 40+ hours per week to the project throughout the summer. I will be residing in Kerala, India, throughout the GSoC coding period.

- In case of any unforeseen events, additional hours will be utilized to compensate and ensure the project stays on track.
- Any such situations will be communicated to the mentor in advance so the timeline can be adjusted collaboratively without affecting overall progress.

Open Source Contributions

New to open source contributions, but not entirely unfamiliar with the ecosystem. I built and published typify.js (an open-source, lightweight auto-typing animation library), which gave me hands-on experience with **semantic versioning**, **pull requests**, **writing open-source documentation**, and **publishing to package registries** (NPM).

My open source contributions journey began during this GSoC period with contributions to openfoodfacts-explorer, open-prices-frontend, and the openfoodfacts-server.

Pull Request	Repo	Status
fix(ui): truncate overflowing usernames in Data Sources panel	openfoodfacts-explorer	Merged
feat: add Matomo event for personal search toggle (#8191)	openfoodfacts-server	Merged

feat: add keyboard shortcuts for product pages (#656)	openfoodfacts-explorer	Under review
feat: added search functionality in Countries Page	open-prices-frontend	Under review

I intend to remain an active member of the Open Food Facts community, providing meaningful improvements and maintenance well beyond the GSoC period.

Prototyping Experience

1. **DuckDB Full Text & Vector Search**: built a search pipeline on the OFF Canada dataset with BM25 Full Text Search and Sentence Transformer Vector Search using array_cosine_similarity.
2. **CLIP + Qdrant Image Search**: built a cross-modal image and text similarity system on OFF data.
3. **API Call Interceptor**: prototyped a network request interceptor in the content script to capture store product IDs and barcodes from grocery site API calls.
4. **IntersectionObserver Lazy Loading**: confirmed viewport-based API triggering works reliably on real grocery site DOM structures.
5. **Custom i18n Service**: validated a lightweight EN/FR translation manager as a leaner alternative to native browser i18n and third-party libraries.
6. **Orchestrator & Adapter Pattern**: explored multiple approaches and architectures and decided to go with this pattern.
7. **Preact + TailwindCSS**: experimented with them to avoid using plain CSS and JS string-based templating.
8. **Preference Score Evaluator**: built and tested the full scoring system using attribute_groups values.

Demo & Repo: openfoodfacts-gsoc2026-prototype

Relevant Past Projects

- FoodSnap - food identification and calorie estimation using Python and image recognition. Demonstrates my ability to build Python-based APIs and develop food item-matching solutions.
- PixelPeep - image similarity search using perceptual hashing. Helped me think about image-based product matching and its possibilities.
- Currency Converter Extension - a real-time currency converter browser extension built with JavaScript, integrating a live currency API. This shows my ability to work with browser extensions and api integrations.